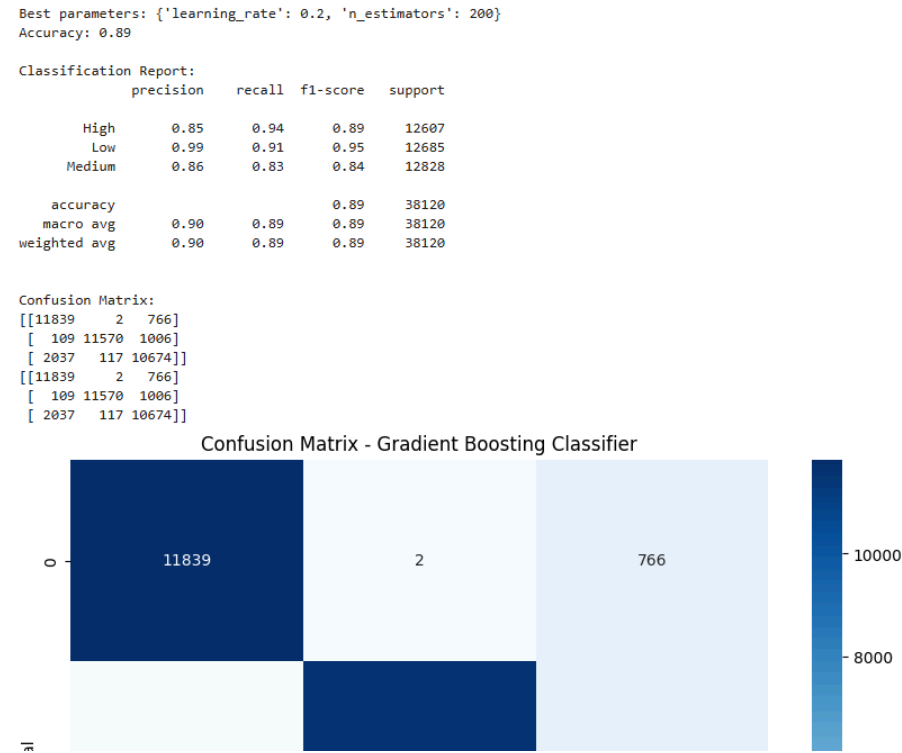


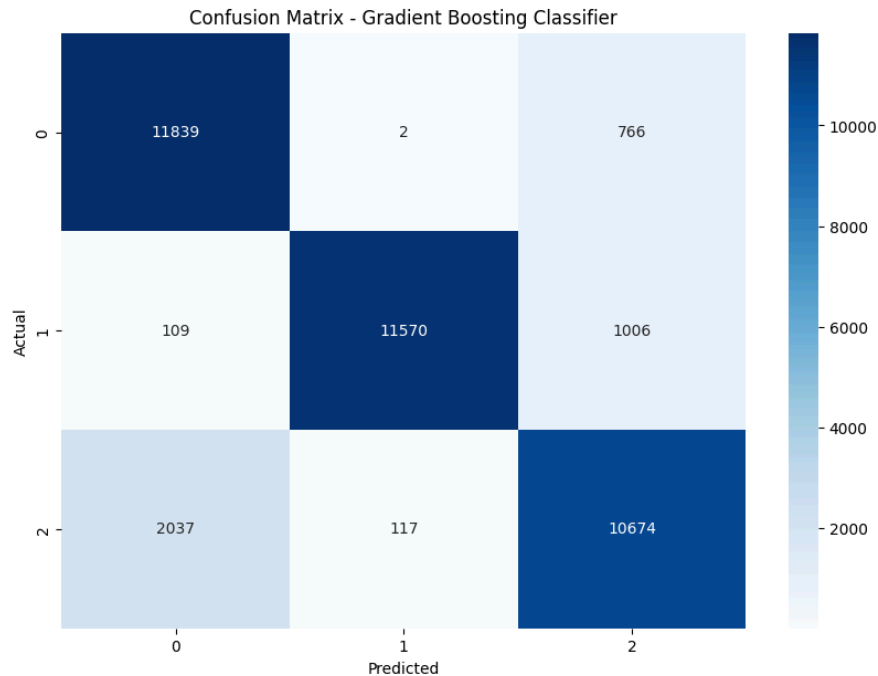
This document contains comprehensive explanations for Model 2:

Initially, I used a Gradient Boosting classifier without GridSearch as the primary model. The results were as follows:

Accuracy: 0.89				
Classification Report:				
	precision	recall	f1-score	support
High	0.84	0.94	0.89	12607
Low	0.99	0.91	0.95	12685
Medium	0.86	0.82	0.84	12828
accuracy			0.89	38120
macro avg	0.90	0.89	0.89	38120
weighted avg	0.90	0.89	0.89	38120
Confusion Matrix:				
[[11855 19 733]				
[104 11589 992]				
[2138 141 10549]]				

Afterward, I attempted to enhance the model using GridSearch to identify the optimal parameters:





High Class: In the intricate dance of data, our model demonstrates a keen eye for identifying High instances, boasting an impressive recal

Following that, I experimented with Random Forest, Logistic Regression, and XGBoost. I compared the performance of these models without utilizing any optimization tools like GridSearch:

```
#training and evaluating models
# Random Forest
rf = RandomForestClassifier(random_state=42)
rf.fit(X_train_scaled, y_train)
y_pred_rf = rf.predict(X_test_scaled)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
print(f"Random Forest Accuracy: {accuracy_rf:.2f}")

# xgBoost
xgb = XGBClassifier(random_state=42)
xgb.fit(X_train_scaled, y_train)
y_pred_xgb = xgb.predict(X_test_scaled)
accuracy_xgb = accuracy_score(y_test, y_pred_xgb)
print(f"xgBoost Accuracy: {accuracy_xgb:.2f}")

#Logistic regression
log_reg = LogisticRegression(random_state=42, max_iter=1000)
log_reg.fit(X_train_scaled, y_train)
y_pred_log_reg = log_reg.predict(X_test_scaled)
accuracy_log_reg = accuracy_score(y_test, y_pred_log_reg)
print(f"Logistic Regression Accuracy: {accuracy_log_reg:.2f}")
```

Random Forest Accuracy: 0.94
XGBoost Accuracy: 0.90
Logistic Regression Accuracy: 0.57

Apparently the best result gives the Random Forest which Spyros is developing more but I just wanted to check what result it would give to me. The idea of this code above is to compare the three models and if I can improve them. I will actually try below more iterations for XGBoost because it seems it would have good results.

It appears that RandomForest yielded the highest accuracy (which led us to select it as our final model). However, I also observed strong performance from XGBoost, prompting further iterations on this model.

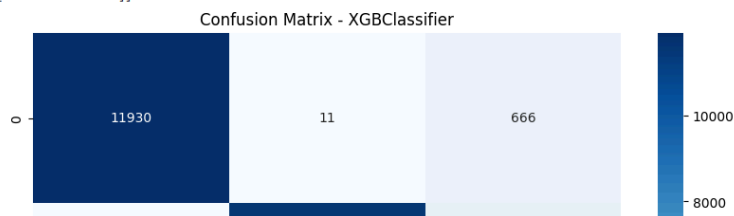
```
Best parameters: {'colsample_bytree': 1.0, 'gamma': 0, 'learning_rate': 0.1, 'max_depth': 4, 'min_child_weight': 2, 'n_estimators': 100, 'subsample': 0.8}
Accuracy: 0.89
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.84         0.95         0.89       12607
     1       0.99         0.91         0.95       12685
     2       0.86         0.82         0.84       12828

 accuracy          0.90
 macro avg          0.90         0.89         0.89       38120
weighted avg          0.90         0.89         0.89       38120
```

```
Confusion Matrix:
[[11930  11  666]
 [ 100 11531 1054]
 [ 2169  117 10542]]
[[11930  11  666]
 [ 100 11531 1054]
 [ 2169  117 10542]]
```



Following my experimentation with XGBoost, I observed superior results and proceeded to further experiment with advanced feature engineering, particularly focusing on polynomial features.

```
Accuracy with Polynomial Features: 0.92
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.88         0.98         0.92       12607
     1       0.99         0.92         0.95       12685
     2       0.90         0.86         0.88       12828

 accuracy          0.92
 macro avg          0.92         0.92         0.92       38120
weighted avg          0.92         0.92         0.92       38120
```

```
Confusion Matrix:
[[12315  2  290]
 [ 77 11674  934]
 [ 1665 103 11060]]
[[12315  2  290]
 [ 77 11674  934]
 [ 1665 103 11060]]
```



The accuracy improved from 90% to 92%, showcasing a significant enhancement in the model's performance. This achievement is noteworthy as it demonstrates the comprehensive lifecycle of the model, from its inception to refinement.

Following this, I attempted to combine the best-performing models thus far: Random Forest, XGBoost, and Gradient Boosting. My aim was to ascertain the collective accuracy of these models when merged:

Combining models Random Forest and XGBoosting and Gradient Boosting

```
9]: from sklearn.ensemble import VotingClassifier
import seaborn as sns

# Define individual models
model1 = RandomForestClassifier(random_state=42)
model2 = XGBClassifier(random_state=42)
model3 = GradientBoostingClassifier(random_state=42)

# combining models in a voting classifier
voting_clf = VotingClassifier(estimators=[
    ('rf', model1),
    ('xgb', model2),
    ('gbc', model3)
], voting='soft')

#training the voting classifier
voting_clf.fit(X_train_scaled, y_train)

# predictions
y_pred_voting = voting_clf.predict(X_test_scaled)

# evaluation of the model
accuracy_voting = accuracy_score(y_test, y_pred_voting)
print(f"Voting Classifier Accuracy: {accuracy_voting:.2f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_voting))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred_voting))

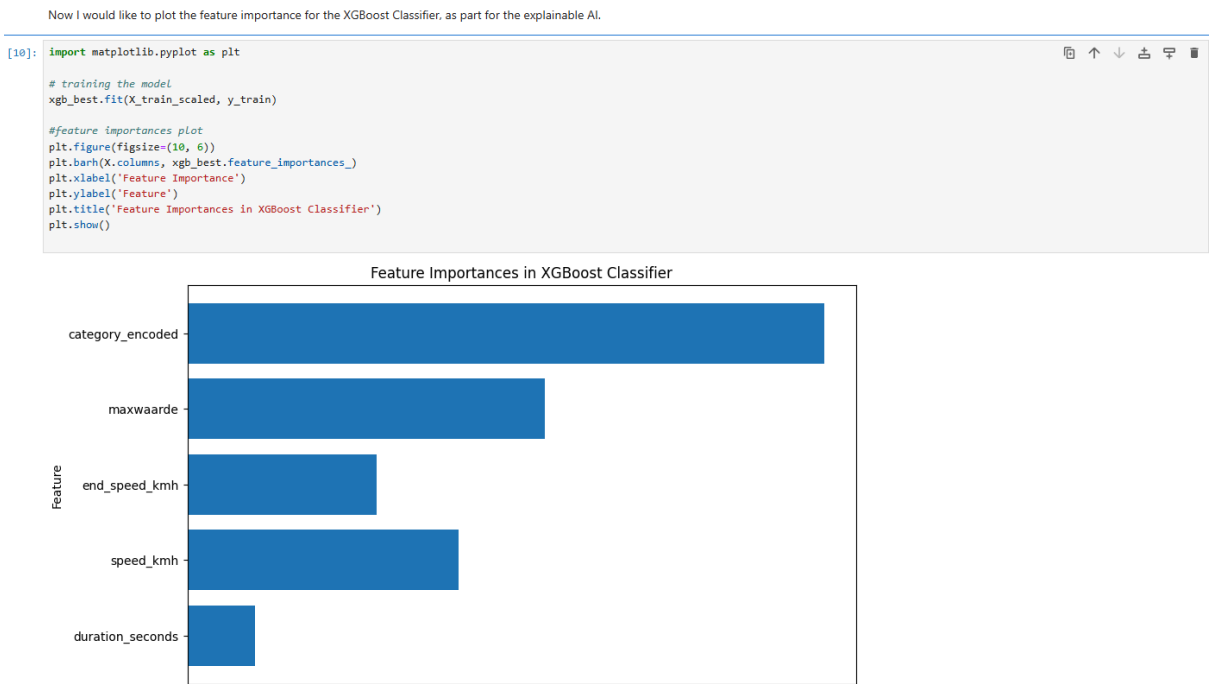
cm = confusion_matrix(y_test, y_pred_best)
print(cm)

# Plot confusion matrix
plt.figure(figsize=(10, 7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - XGBClassifier')
plt.show()
```

So far, this was the best performing model with 93% accuracy.

Voting Classifier Accuracy: 0.93					
Classification Report:					
	precision	recall	f1-score	support	
0	0.89	0.99	0.94	12607	
1	0.99	0.92	0.96	12685	
2	0.92	0.88	0.90	12828	
accuracy			0.93	38120	
macro avg	0.93	0.93	0.93	38120	
weighted avg	0.93	0.93	0.93	38120	

This is my last step in this notebook where I used XAI to explain the feature importance for the XGBoost:



In conclusion, I can affirm that the best results from all iterations yielded a commendable accuracy of approximately 90%.